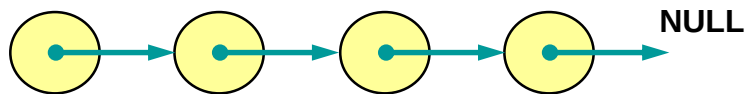


Лекция 8

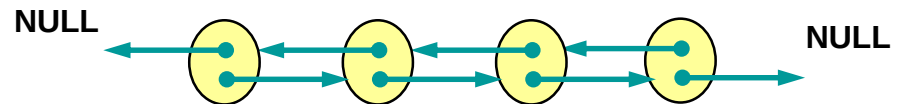
Динамические структуры данных (часть 1): односвязный линейный список

1 Виды списков

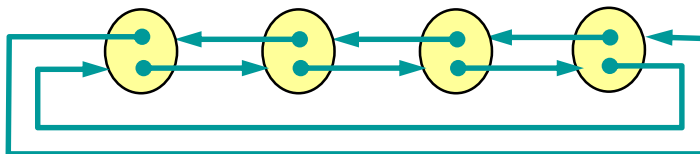
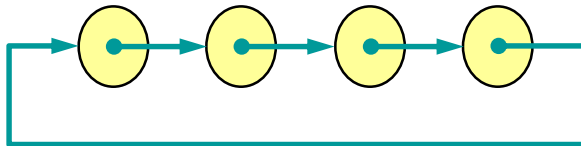
односвязный



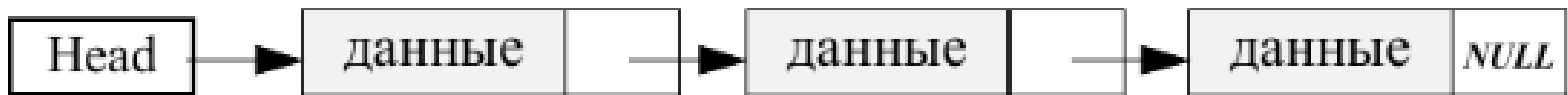
двунаправленный (двусвязный)



циклические списки (кольца)



2 Односвязный линейный список



данные — информационная часть звена списка (величина любого простого или структурированного типа, кроме файлового),

-> — указатель на следующее звено списка;

Head — указатель на заглавное звено списка.

Представление узла списка в виде структуры

```
struct Node
{
    int Data; // поле данных
    Node *Next; // указатель на следующий элемент
};
```



Указатель на начало списка

```
Node *Head=NULL;
```

Основные действия, производимые над элементами списка:

- ✓ Добавление узла в список (в начало, в конец, в заданную позицию)
- ✓ Удаление узла из списка (из начала, из конца, из заданной позиции)
- ✓ Вывод элементов списка

Создание узла

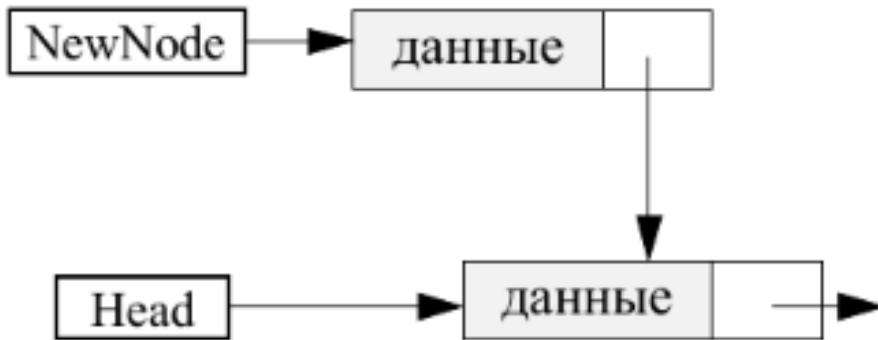
```
struct Node
{
    int Data; // поле данных
    Node *Next; // указатель на следующий элемент
};

// создание узла
// d - данные узла
Node* CreateNode (int d)
{
    // указатель на новый узел
    Node * NewNode = new Node;
    // заполнение данными
    NewNode->Data = d;
    NewNode->Next = NULL;
    return NewNode;
}
```

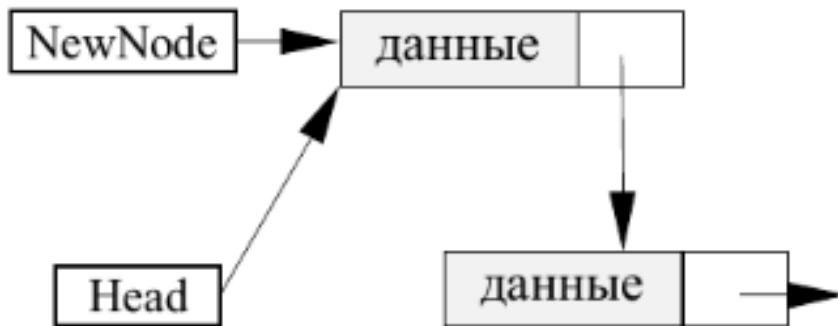


Добавление узла в начало списка

- 1) создать новый узел NewNode;
- 2) установить ссылку узла NewNode на голову существующего списка;



- 3) установить голову списка на новый узел.



```
// добавление узла в начало списка
Node * AddNodeStart(Node *Head, int d)
{
    // создание нового узла
    Node * NewNode = CreateNode (d);
    // установить ссылку на голову списка
    NewNode -> Next = Head;
    // установить голову списка на новый узел
    return NewNode;
}
...
Node *Head=NULL;
Head = AddNodeStart(Head, 1);
```

Печать данных списка

- 1) установить новый указатель на начало списка;
- 2) выводить данные текущего узла и переходить на следующий узел пока текущий элемент списка существует (указатель - не NULL).

```
// печать списка
void SpisokPrint(Node *Head)
{
    Node *p;

    p = Head;
    do {
        printf("%d ", p->Data); // вывод значения элемента p
        p = p->Next; // переход к следующему узлу
    } while (p != NULL);
}
```

Добавление узлов в начало списка и печать списка

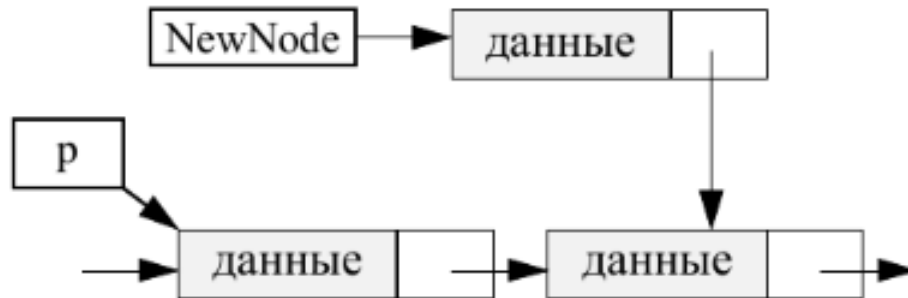
```
1  #include <iostream>
2
3  using namespace std;
4
5  struct Node
6  {
7      int Data; // поле данных
8      Node *Next; // указатель на следующий элемент
9  };
10
11  // создание узла
12  // d - данные узла
13  Node* CreateNode (int d)
14  {
15      // указатель на новый узел
16      Node * NewNode = new Node;
17      // заполнение данными
18      NewNode->Data = d;
19      NewNode->Next = NULL;
20      return NewNode;
21  }
22
23  // печать списка
24  void SpisokPrint(Node *Head)
25  {
26      Node *p;
27      p = Head;
28      do {
29          printf("%d ", p->Data); // вывод значения элемента p
30          p = p->Next; // переход к следующему узлу
31      } while (p != NULL);
32  }
33
34  // добавление узла в начало списка
35  Node * AddNodeStart(Node *Head, int d)
36  {
37      // создание нового узла
38      Node * NewNode = CreateNode (d);
39      // установить ссылку на голову списка
40      NewNode -> Next = Head;
41      // установить голову списка на новый узел
42      return NewNode;
43  }
44
45  int main()
46  {
47      Node *Head=NULL;
48
49      Head = AddNodeStart(Head, 1);
50      Head = AddNodeStart(Head, 2);
51      Head = AddNodeStart(Head, 3);
52
53      SpisokPrint(Head);
54      return 0;
55  }
```

3 2 1

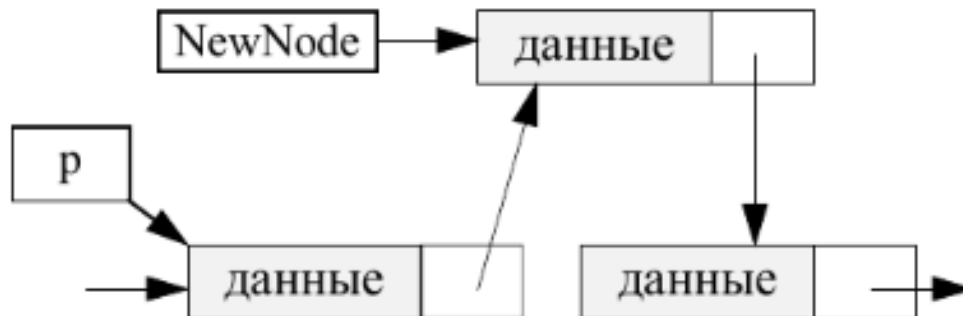
Добавление узла после заданного

Дан адрес *NewNode* нового узла и адрес *p* одного из существующих узлов в списке. Требуется вставить в список новый узел после узла с адресом *p*. Эта операция выполняется в два этапа:

1) установить ссылку нового узла на узел, следующий за данным;



2) установить ссылку данного узла *p* на *NewNode*.



```

45 // добавление узла после заданного
46 void AddNodeAfter(Node *p, Node *NewNode)
47 {
48     NewNode->Next = p->Next;
49     p->Next = NewNode;
50 }
51
52 int main()
53 {
54     Node *Head=NULL;
55
56     Head = AddNodeStart(Head, 1);
57     Head = AddNodeStart(Head, 2);
58     Head = AddNodeStart(Head, 3);
59
60     // p указывает на 2ой узел
61     Node *p=Head->Next;
62     // создание нового узла
63     Node * NewNode = CreateNode (4);
64     // вставка нового узла после 2ого узла
65     AddNodeAfter(p, NewNode);
66
67     SpisokPrint(Head);
68     return 0;
69 }

```

3 2 4 1

Добавление узла перед заданным?
Добавление узла в конец списка?

Удаление узла



```
52 // удаление узла DelNode
53 void DeleteNode(Node *DelNode, Node * Head)
54 {
55     Node *q = Head;
56     // если первый элемент DelNode
57     if (Head == DelNode) Head = DelNode->Next;
58     else { // ищем элемент, который ссылается на DelNode
59         while (q && q->Next != DelNode) q = q->Next;
60         if ( q == NULL) return;
61         q->Next = DelNode->Next; }
62     delete DelNode;
63 }
64
65 int main()
66 {
67     Node *Head=NULL;
68     Head = AddNodeStart(Head, 1);
69     Head = AddNodeStart(Head, 2);
70     Head = AddNodeStart(Head, 3);
71
72     // p указывает на 2ой узел
73     Node *p=Head->Next;
74     // создание нового узла
75     Node * NewNode = CreateNode (4);
76     // вставка нового узла после 2ого узла
77     AddNodeAfter(p, NewNode);
78     // удаление второго узла
79     DeleteNode(p, Head);
80     SpisokPrint(Head);
81     return 0;
82 }
```

3 4 1

Удаление первого узла?
Удаление последнего узла?

Реализация односвязного линейного списка как класса

```
1  #include <iostream>
2  using namespace std;
3  //////////////////////////////////////
4
5  struct Node // элемент списка
6  {
7      int Data; // некоторые данные
8      Node* Next; // указатель на следующую структуру
9  };
10
11  class List // список
12  {
13  private:
14      Node* Head;
15  public:
16      List() { Head = NULL; }
17      void AddNode(int d); // добавление элемента
18      void SpisokPrint(); // показ данных
19  };
20
21  // добавление элемента
22  void List::AddNode(int d)
23  {
24      Node* NewNode = new Node;
25      NewNode->Data = d;
26      NewNode->Next = Head;
27      Head = NewNode;
28  }
```

```
29
30  // показ данных
31  void List::SpisokPrint()
32  {
33      Node* current = Head;
34      // пока есть данные
35      while(current)
36      {
37          // печатаем
38          cout << current->Data << endl;
39          // движемся к следующему элементу
40          current = current->Next;
41      }
42  }
43
44  int main()
45  {
46      List L; // создаем список
47      L.AddNode(1);
48      L.AddNode(2);
49      L.AddNode(3);
50
51      L.SpisokPrint(); // показываем список
52      return 0;
53  }
```

3

2

1

Представление элемента односвязного линейного списка как класса

```
1  #include <iostream>
2  #include <string>
3  using namespace std;
4
5  class People    // элемент списка
6  {
7      string Name;
8      int age;
9      People* Next;
10
11  public:
12      void SetData(string n, int a, People* p)
13      {Name = n; age = a; Next = p;}
14      People* GetNext(){return Next;}
15      void Print(){ cout<<Name<<" "<<age<<endl; }
16  };
17
18  class List    // список
19  {
20  private:
21      People* Head;
22  public:
23      List() { Head = NULL; }
24      void AddNode(string n, int a); // добавление элемента
25      void SpisokPrint(); // показ данных
26  };
27
```

```
Bob 25
Max 15
Alex 5
```

```
28 // добавление элемента
29 void List::AddNode(string n, int a)
30 {
31     People* NewPeople = new People;
32     NewPeople->SetData(n,a,Head);
33     Head = NewPeople;
34 }
35
36 // показ данных
37 void List::SpisokPrint()
38 {
39     People* current = Head;
40     // пока есть данные
41     while(current)
42     {
43         // печатаем
44         current->Print();
45         // двигаемся к следующему элементу
46         current = current->GetNext();
47     }
48 }
49
50 int main()
51 {
52     List L; // создаем список
53     L.AddNode("Alex", 5);
54     L.AddNode("Max", 15);
55     L.AddNode("Bob", 25);
56
57     L.SpisokPrint(); // показываем список
58     return 0;
59 }
```